Function-Aware Fill-in-the-Middle as Mid-Training for Coding Agent Foundation Models

Anonymous Author(s)
Affiliation
Address
email

Abstract

Coding agents must integrate external tool returns into ongoing reasoning—a capability that standard left-to-right pretraining on code exposes only in its forward direction. We observe that the *action* $\rightarrow$ *observation* $\rightarrow$ *continuation* loop of a coding agent is structurally isomorphic to a function call site, where a caller binds arguments, a callee returns a value computed elsewhere, and downstream code consumes that value. This conditioning structure exists at internet scale in ordinary code. We exploit it through *function-aware fill-in-the-middle (FIM) mid-training*: a self-supervised objective that masks functions selected via program dependency graph analysis and a complexity–inferability double criterion. We mid-train Qwen2.5-Coder (7B/14B/32B) and Qwen3-8B on a 3B-token decontaminated corpus drawn from 990 GitHub repositories, then apply existing agentic post-training pipelines (R2E-Gym, SWE-Smith) without modification. Mid-training improves SWE-Bench-Verified by $+2.8/+3.0/+3.3$ points at 7B/14B/32B—a consistent gain across scales—and the improvement holds across post-training pipelines and base-model families. Beyond in-domain gains, mid-training also mitigates the capability erosion that agentic post-training otherwise inflicts on non-agent coding (e.g., LiveCodeBench) and non-coding tool-use benchmarks (tau-bench, BFCL): although the mid-training corpus contains Python code only, the function-call inductive bias survives post-training and yields consistent improvements over post-training alone.

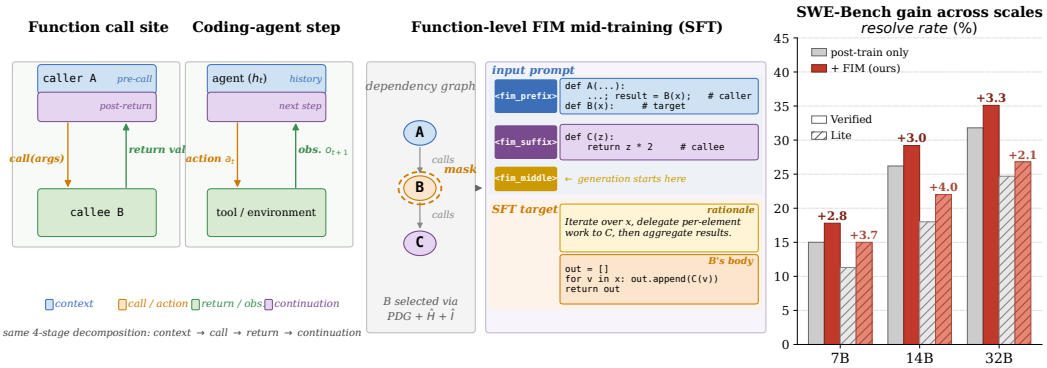

Figure 1: **Left:** A function call site and a single step of a coding agent are *structurally similar*, decomposing into the same four stages: context, call/action, return/observation, continuation. **Middle:** We exploit this analogy via function-aware FIM mid-training. A function B is selected from the program dependency graph using complexity ($\hat{H}$) and inferability ($\hat{I}$) scores; the model is then fine-tuned to produce a chain-of-thought rationale followed by B's body, given the surrounding file as an FIM-formatted prompt. **Right:** Mid-training yields consistent gains across the Qwen-2.5-Coder model series on both SWE-Bench-Verified (solid bars) and SWE-Bench-Lite (hatched bars).

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1 Introduction

Coding agents that resolve real software engineering issues have moved from research demos to deployed systems [10, 21, 18]. Their progress, however, has been driven almost entirely by scaling synthetic agent-trajectory data during post-training: pipelines such as SWE-Gym [14], R2E-Gym [9], and SWE-Smith [22] curate or synthesize trajectories that imitate human or LLM behavior on issue-resolution tasks. The base model these pipelines start from is typically a code LLM trained with next-token prediction (and, in some cases, random-span FIM) on internet-scale code [7, 5, 11]. Between these two stages lies a training-time gap: the base model is rarely optimized for the conditioning structure that agentic post-training will later demand. We treat this gap as an opportunity for a dedicated *mid-training* stage that aligns the base model with agent-relevant inductive biases before agent-specific data is introduced.

Our central observation is that the inductive bias required by a coding agent already exists in ordinary code, but in a shape that left-to-right pretraining systematically under-exposes. At each step an agent maintains a history h_t, samples an action $a_t \sim \pi(a_t \mid h_t)$, receives an observation o_{t+1} produced by an external process, and continues conditioned on the entire trace. This four-part decomposition—context, action, externally-computed return, continuation—is precisely the decomposition of a function call site: pre-call code that establishes intent and binds arguments; the call itself; a return value produced by code outside the immediate scope; and downstream code that consumes the return value (Figure 1, left). A model trained to reason *bidirectionally* about function-level dependencies must learn to reconstruct a callee's behavior from caller context and downstream usage, which is the same competence required to predict an agent's continuation given a history and a tool return. The correspondence is structural rather than literal—FIM training conditions on a given suffix while agent inference generates one—but it suggests that representations induced by the former should transfer to the latter, an empirical question we address in Section 3.

The fill-in-the-middle objective itself is not new, dating back to early work on code LLMs [1]; recent models mix random-span FIM into their pretraining corpora [7, 5, 11]. We argue, however, that random-span FIM is poorly aligned with the agentic conditioning structure for three reasons. *(i) Span boundaries are syntactically arbitrary:* most masked spans cut through expressions or partial statements and provide only weak signal about function-level dependencies. *(ii) There is no reasoning supervision:* the model fills the span directly, with no intermediate rationale mirroring an agent's think-then-act pattern. *(iii) The objective is dissolved into pretraining:* by the time agent post-training begins, any structural prior conferred by FIM has been amortized across trillions of unrelated tokens. We address all three points. We select masking targets at function granularity using program dependency graph analysis combined with a complexity–inferability double criterion that is deliberately base-model-agnostic (Section 2.3). We embed chain-of-thought rationales *inside* the FIM middle span so that the model learns to produce a reasoning trace consistent with the eventual code (Section 2.4). And we apply the objective at a dedicated mid-training stage, concentrating its signal immediately before agentic post-training rather than spreading it across pretraining.

We evaluate this recipe along three axes of robustness. *Across model sizes*, mid-training improves SWE-Bench-Verified by $+2.8$, $+3.0$, and $+3.3$ points at 7B, 14B, and 32B, respectively: the gain is consistent across the scales we evaluate, indicating that the structural prior is not absorbed by larger pretrained models and remains useful in the practical deployment range. *Across post-training pipelines*, mid-training improves both R2E-Gym and SWE-Smith on the same 7B base, with the SWE-Smith pairing yielding $+5.3$ points on SWE-Bench-Verified. *Across base-model families*, mid-training transfers from Qwen2.5-Coder to Qwen3-8B with a $+3.2$-point gain on SWE-Bench-Verified (paired with the SWE-Lego post-training pipeline), suggesting the prior is not specific to a single pretraining recipe.

A second set of results probes our motivating hypothesis. We note first that all benchmarks in this paper are evaluated on checkpoints that have undergone the full pipeline—either post-training alone (baseline) or mid-training followed by post-training (ours)—because a mid-trained checkpoint without subsequent instruction-style post-training has degraded instruction-following ability and is not directly comparable to instruction-tuned baselines. Under this protocol, we observe that agentic post-training alone substantially regresses non-target capabilities at 14B: it drops LiveCodeBench, BFCL, and tau-bench by double-digit margins in some cases. Adding mid-training before the same post-training largely restores these capabilities, recovering $+11.1$ points on LiveCodeBench and gaining $+2.5$ on BFCL and $+4.0$ on tau-bench over post-training alone. Because the mid-training

corpus contains only Python code with no tool-use data, mid-training a priori has no reason to affect tau-bench or BFCL; the fact that it does, and in the same direction as the in-domain SWE-Bench gains, provides direct evidence for the isomorphism between function calls and agent tool calls.

Contributions. **(1) Conceptual framing.** We articulate a structural correspondence between function call sites and the action–observation–continuation loop of coding agents, and motivate function-granularity FIM as a self-supervised prior for agent capability that exists at internet scale in ordinary code. **(2) Method.** We introduce a function-aware FIM mid-training pipeline combining program dependency graph analysis, a hand-designed complexity–inferability double criterion that is base-model-agnostic, and chain-of-thought rationales embedded inside the FIM middle span. **(3) Three-axis robustness.** We empirically validate the recipe along three independent axes—model size, post-training pipeline, and base-model family—observing consistent in-domain gains in every configuration. **(4) Direct test of the motivating hypothesis.** The same Python-only mid-training corpus—which contains no tool-use data—also yields gains on tau-bench, BFCL, and LiveCodeBench, providing direct evidence that the function-call inductive bias survives agentic post-training and transfers across task families. **(5) Open release.** We release the 990-repository decontaminated corpus (78K filtered self-contained Python files, $\approx 33\mathrm{M}$ lines), the full selection pipeline, and all mid-training checkpoints to support reproduction.

2 Method

2.1 Motivation: Function Calls as Agent-Like Structures

Formally, at step t a coding agent maintains a history h_t and samples an action $a_t \sim \pi(a_t \mid h_t)$; the environment yields an observation $o_{t+1} \sim p(o_{t+1} \mid h_t, a_t)$; and the agent continues conditioned on the full trace, $\pi(\cdot \mid h_t, a_t, o_{t+1})$. Under the function-call/agent-step correspondence introduced in Section 1 (Figure 1, left), h_t aligns with the pre-call context, a_t with the call, o_{t+1} with the return, and the continuation with downstream usage of the returned value.

This correspondence motivates our central design: extract an agentic conditioning signal from the structure already present in source code via the fill-in-the-middle (FIM) objective. The correspondence is *structural* rather than literal—FIM conditions on a given suffix while agent inference generates one—but training a model to reason bidirectionally about function-level dependencies should induce a representation useful when the "suffix" is generated rather than observed. Standard random-span FIM [7, 5, 11] captures this structure only incidentally; we instead construct a *function-aware* variant that selects masking targets based on program structure and contextual predictability.

2.2 Data Collection and Decontamination

We curate a corpus of 990 Python repositories from GitHub. Starting from $\sim 2{,}000$ candidates retrieved by combining a star-count threshold with ten topic queries, we apply manual quality filtering and remove every repository whose origin overlaps with the source repositories of SWE-Bench [10] (verified by repository name and any known fork). To eliminate the possibility of test-time leakage from training-data commits, we restrict each repository to commits whose timestamp precedes the earliest base-commit used in SWE-Bench-Verified and SWE-Bench-Lite. The filtered corpus contains $\approx 78\mathrm{K}$ self-contained Python files ($\approx 33\mathrm{M}$ lines) and yields roughly $3\mathrm{B}$ training tokens under the Qwen2.5-Coder tokenizer. Topic-category breakdown, per-property statistics, license inventory, and commit hashes are reported in Appendix A; the full repository list will be released to support reproduction.

2.3 Function-Aware FIM Target Selection

Given the file source of each repository, our pipeline produces a set of mask targets together with the corresponding masked file. Each target is a single function or a connected group of 2–3 functions; the latter variant is studied in Section 3.4. The pipeline has four stages: dependency-graph construction, complexity scoring, inferability scoring, and threshold-based selection. Figure 2 traces all four stages on a deliberately small example—a calculator class with two top-level helpers—which we will use as a running example throughout this subsection. The full single-function selection algorithm is given in

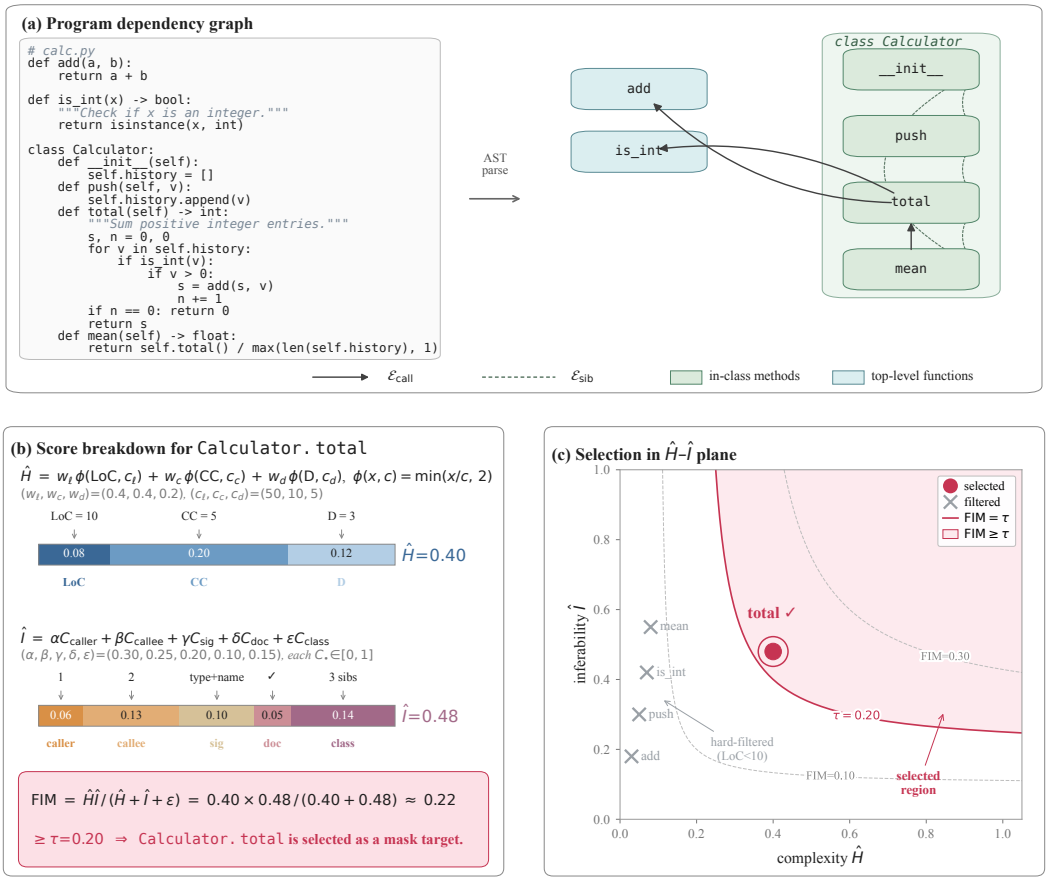

Figure 2: Function-aware FIM target selection illustrated on a small example file (a calculator class with two top-level helpers).

Appendix B.1; every quantity shown in Figure 2(b)—including the line-by-line counts that produce LoC = 10, CC = 5, D = 3, and the five inferability components—is derived explicitly in Appendix **??**.

2.3.1 Program Dependency Graph

For each source file we parse its AST and extract the set $\mathcal{V}$ of function nodes (top-level functions and class methods, identified by qualified names). We construct two edge sets: *call edges* $\mathcal{E}_{\text{call}}$ between caller and callee, and *sibling edges* $\mathcal{E}_{\text{sib}}$ between methods of the same class (capturing intra-class coupling that does not manifest as direct calls but carries information through shared instance state). Call resolution handles common Python idioms (direct invocations, class instantiation, `self`/`cls` method calls) with a short-name fallback against the qualified-name index; details are in Appendix B.2. Figure 2(a) shows both edge types on the running example: `total` reaches the top-level helpers via $\mathcal{E}_{\text{call}}$, while $\mathcal{E}_{\text{sib}}$ connects the four `Calculator` methods that share `self.history`.

2.3.2 Complexity Score $\hat{H}$

For each $v \in \mathcal{V}$ we define

$$\hat{H}(v) \;=\; w_\ell\, \phi(\text{LoC}(v),\, c_\ell) + w_c\, \phi(\text{CC}(v),\, c_c) + w_d\, \phi(\text{D}(v),\, c_d)\,, \tag{1}$$

where $\text{LoC}(v)$ is lines of code, $\text{CC}(v)$ is McCabe cyclomatic complexity, $\text{D}(v)$ is the maximum nesting depth of control-flow constructs, and $\phi(x, c) = \min(x/c,\, 2)$ normalizes each quantity by a soft cap of 2 (allowing genuinely complex functions to score above the median without letting outliers dominate). Caps and weights are listed in Appendix B.3. The top bar of Figure 2(b) shows how

the three weighted components combine into $\hat{H}(\texttt{total}) = 0.40$ on the running example, with the hyperparameter values annotated directly below the formula and the underlying line/branch counts derived in Appendix **??**.

2.3.3 Inferability Score $\hat{I}$

A target should be *recoverable* from the surrounding context. $\hat{I}(v)$ aggregates five context-derived signals that approximate the mutual information between v's body and the rest of the file:

$$\hat{I}(v) \;=\; \alpha\, C_{\text{caller}}(v) + \beta\, C_{\text{callee}}(v) + \gamma\, C_{\text{sig}}(v) + \delta\, C_{\text{doc}}(v) + \varepsilon\, C_{\text{class}}(v). \tag{2}$$

C_{caller} scores call-site argument specificity, C_{callee} counts intra-file functions called by v, C_{sig} aggregates type annotations and name descriptiveness, C_{doc} indicates docstring presence, and C_{class} counts in-class siblings; full component formulas and weights are in Appendix B.4. Each component is a hand-designed proxy rather than a learned predictability score: a learned proxy would couple selection to a particular reference model and complicate the cross-base-model generalization analysis (Section 3.2). The lower bar of Figure 2(b) decomposes $\hat{I}(\texttt{total}) = 0.48$ into these five contributions; the class-context term $\varepsilon\, C_{\text{class}}$ dominates because `total` sits among three siblings sharing the same instance state, while the docstring term contributes the least. A component-by-component derivation of each of the five terms on the running example is given in Appendix **??**.

2.3.4 Single-Function Score

The final score combines $\hat{H}$ and $\hat{I}$ in a harmonic-mean-like form that requires both to be simultaneously high, multiplied by a one-sided difficulty penalty $\rho(\Delta(v))$:

$$\text{FIM}(v) \;=\; \frac{\hat{H}(v)\,\hat{I}(v)}{\hat{H}(v) + \hat{I}(v) + \epsilon} \cdot \rho(\Delta(v))\,, \tag{3}$$

where ϵ is a small constant. The harmonic-mean form forces both $\hat{H}$ and $\hat{I}$ to be large simultaneously, penalizing imbalance; the penalty ρ down-weights targets that remain hard even given full context (which would otherwise be unlearnable noise). Full forms of Δ and ρ, together with the hard filters on length, dunder methods, and minimum scores, are deferred to Appendix B.5. Figure 2(c) makes the geometry of this rule explicit: dashed iso-FIM contours bend toward the diagonal, so a function with high $\hat{H}$ but low $\hat{I}$ (or vice versa) lies far from the threshold curve. On the running example, the four short helpers are removed by the hard filters before scoring, and only `Calculator.total`, with $\text{FIM} \approx 0.22 \geq \tau = 0.20$, enters the selected region.

2.3.5 Multi-Function Group Selection

Real-world code patches frequently span multiple related functions [10], motivating an extension that masks *groups* of $k = 2$ or $k = 3$ structurally connected functions; we use this variant only in the granularity ablation (Section 3.4). A group score $\text{FIM}(G)$ multiplies a coupling term, the harmonic-mean-like product of group-level $\hat{H}(G)$ and $\hat{I}(G)$, and a difficulty penalty, with $\hat{I}(G)$ recomputed under joint masking so that intra-group references cannot inflate the score. Topology patterns (eight in total: caller-callee, co-callee, sibling-coupled, mutual-call, call-chain, hub, fan-in, class-triad), the full equations, and selection thresholds are given in Appendix B.6.

2.4 Chain-of-Thought Augmentation

For each selected target we generate a natural-language rationale via Gemini-3-Preview prompted with the masked file and the original function body (template in Appendix B.7). At training time, the rationale is embedded *inside* the FIM middle span, immediately before the function body:

<fim_prefix> ⟨prefix⟩ <fim_suffix> ⟨suffix⟩ <fim_middle> ⟨rationale⟩ ⟨body⟩

The model is therefore trained to produce, in order, a reasoning trace followed by code consistent with that trace—mirroring the think-then-act structure of an agent step. The role of the rationale is to provide a structured intermediate supervision aligned with the FIM objective rather than to distill the teacher's generic capability; we isolate this in Section 3.4 via a *self-CoT* variant in which the model under training generates its own rationales, and find that the FIM structure alone already accounts for a substantial fraction of the gain.

3 Experiments

3.1 Setup

Benchmarks. We evaluate on three groups of benchmarks. *Coding agent benchmarks* comprise SWE-Bench-Verified (500 instances) and SWE-Bench-Lite (300 instances) [10]; both are in-domain for the post-training pipelines and constitute our primary measure of end-task progress. *Non-agent coding benchmarks* cover LiveCodeBench [8], OJBench [20], and FullStackBench-EN [2]; these probe pure code generation without an agent loop and serve as a regression check for capability erosion. *Tool-use and out-of-domain agent benchmarks* include Terminal-Bench [12], τ-bench [23], and BFCL [15]; the latter two contain no Python code-editing trajectories at all, and they are the appropriate stage on which to test our central hypothesis that the function-call inductive bias transfers across task families.

Mid-training and post-training pipeline. We mid-train Qwen2.5-Coder-Instruct (7B, 14B, 32B) and Qwen3-8B on the selected FIM corpus. The training objective is the standard FIM loss on the middle span (rationale plus body), packed to the model's native context length and using the model's native FIM sentinel tokens to preserve compatibility with any FIM signal already learned during pretraining. After mid-training we apply existing agentic post-training pipelines without modification: R2E-Gym [9] or SWE-Smith [22] for the Qwen2.5-Coder runs, and SWE-Lego [17] for Qwen3-8B. *Important deviation for Qwen3-8B*: we train for 2 epochs rather than the 4 epochs used in the original SWE-Lego setting, as the latter exhibited noticeable overfitting in our setup. Hyperparameters, token budgets, and learning-rate schedules are reported in Appendix **??**.

Evaluation protocol. All numerical results are reported as the mean over three independent evaluation seeds; the "%" symbol is omitted inside table cells. Standard-deviation bands shown in red ($\pm$?) are placeholders awaiting final replication runs. Results are obtained on the *final* checkpoint of the training pipeline. A "baseline" row refers to the base model followed by an agentic post-training pipeline, while "ours" refers to the same base followed by FIM mid-training and then by the identical post-training pipeline. We do not directly evaluate mid-training-only checkpoints, because FIM data without subsequent instruction-style post-training degrades instruction-following ability and makes any comparison against instruction-tuned baselines unfair on instruction-following benchmarks. Consequently, every gain we attribute to mid-training should be read as a gain that survives subsequent post-training. The agent harness is fixed by the post-training pipeline: R2E-Gym runs use the R2E-Gym scaffold (an OpenHands fork) [18, 9]; SWE-Smith runs use SWE-agent [21]; the Qwen3-8B runs use OpenHands directly, because the SWE-Lego post-training data exceeds the 32K context length of the R2E-Gym/SWE-Smith scaffolds while remaining within Qwen3's 40,960-token context. Maximum turn limits, decoding temperature, and per-step timeouts are deferred to Appendix **??**.

Models and baselines. For Qwen2.5-Coder-Instruct at 7B / 14B / 32B [7] we report (i) the instruction-tuned base without any agentic training, (ii) post-training with R2E-Gym reproduced under our setup, (iii) the official R2E-Gym numbers (in grey, where available), (iv) our recipe of FIM mid-training followed by R2E-Gym post-training, and at 7B additionally the analogous SWE-Smith [22] variants. For Qwen3-8B we follow the SWE-Lego [17] pipeline; the cross-base-model comparison thus also varies the post-training pipeline, a confound we discuss in Section 3.2.

3.2 Main Results on SWE Agent Benchmarks

Table 1 summarizes the in-domain agent results.

Mid-training delivers consistent gains across model scales. Holding the post-training pipeline fixed at R2E-Gym, FIM mid-training improves SWE-Bench-Verified by $+2.80$ at 7B, $+3.00$ at 14B, and $+3.30$ at 32B. The absolute gain stays within a narrow band as the base model grows by more than $4\times$ in parameters. SWE-Bench-Lite shows the same direction at every scale ($+3.67$ / $+4.00$ / $+2.10$). We do not interpret this as evidence of monotone scaling, but rather as evidence that the structural prior is not absorbed by larger pretrained models in the practical deployment range we evaluate. We do not extrapolate these trends beyond 32B.

Table 1: Main results on coding agent benchmarks (SWE-Bench-Verified and SWE-Bench-Lite). All numbers are percentages. Verified and Lite cells are reported as the mean over three independent seeds with their std band; the *Average* column is the unweighted mean of the Verified and Lite means. Bolded rows are our recipe (FIM mid-training followed by the same post-training pipeline as the row above). Rows tagged *officially reported* are quoted directly from the corresponding original publications, including their std bands; all other rows are reproduced under our setup. For SWE-Lego we post-train for only 2 epochs (whereas the official SWE-Lego release post-trains for 4 epochs) in order to prevent overfitting.

Setting	SWE-Bench-Verified	SWE-Bench-Lite	Average
Qwen2.5-Coder-7B-Instruct			
— (no agentic training)	1.8 ($\pm$1.3)	1.0 ($\pm$1.0)	1.40
+ R2E-Gym [9] (reproduced)	15.00 ($\pm$1.5)	11.33 ($\pm$1.2)	13.17
+ R2E-Gym (officially reported)	19.00 ($\pm$1.0)	11.00 ($\pm$0.8)	15.00
+ FIM-Midtrain + R2E-Gym	**17.80** ($\pm$1.4)	**15.00** ($\pm$1.1)	**16.40**
Δ (ours vs. reproduced)	+2.80	+3.67	+3.24
+ SWE-Smith [22] (reproduced)	12.30 ($\pm$1.2)	14.20 ($\pm$1.4)	13.25
+ SWE-Smith (officially reported)	15.20	11.70	13.45
+ FIM-Midtrain + SWE-Smith	**17.60** ($\pm$1.3)	**14.70** ($\pm$1.0)	**16.15**
Δ (ours vs. reproduced)	+5.30	+0.50	+2.90
Qwen2.5-Coder-14B-Instruct			
— (no agentic training)	4.0 ($\pm$1.6)	2.7 ($\pm$1.0)	3.35
+ R2E-Gym (reproduced)	26.20 ($\pm$1.4)	18.00 ($\pm$1.1)	22.10
+ R2E-Gym (officially reported)	26.80 ($\pm$1.4)	20.67 ($\pm$0.7)	23.74
+ FIM-Midtrain + R2E-Gym	**29.20** ($\pm$1.5)	**22.00** ($\pm$1.2)	**25.60**
Δ (ours vs. reproduced)	+3.00	+4.00	+3.50
Qwen2.5-Coder-32B-Instruct			
— (no agentic training)	7.0 ($\pm$1.3)	3.0 ($\pm$1.4)	5.00
+ R2E-Gym (reproduced)	31.80 ($\pm$1.6)	24.70 ($\pm$1.4)	28.25
+ R2E-Gym (officially reported)	34.40 ($\pm$1.2)	23.77 ($\pm$0.8)	29.09
+ FIM-Midtrain + R2E-Gym	**35.10** ($\pm$1.7)	**26.80** ($\pm$1.3)	**30.95**
Δ (ours vs. reproduced)	+3.30	+2.10	+2.70
Qwen3-8B			
— (no agentic training)	7.60 ($\pm$1.2)	5.80 ($\pm$0.9)	6.70
+ SWE-Lego [17] (reproduced)	31.80 ($\pm$1.0)	27.30 ($\pm$1.1)	29.55
+ FIM-Midtrain + SWE-Lego	**35.00** ($\pm$1.5)	**32.70** ($\pm$1.3)	**33.85**
Δ (ours vs. reproduced)	+3.20	+5.40	+4.30

The improvement transfers across post-training pipelines. Replacing R2E-Gym with SWE-Smith on the same 7B base, FIM mid-training yields a gain of $+5.30$ points on SWE-Bench-Verified, larger than the $+2.80$ observed under R2E-Gym. On SWE-Bench-Lite the SWE-Smith pairing gains only $+0.50$, smaller than the $+3.67$ for the corresponding R2E-Gym pairing; the cross-pipeline transfer is therefore clearer on Verified than on Lite. Taken together, the two pipelines indicate that mid-training is not specifically tuned to a single post-training data distribution, but the magnitude of its benefit can depend on which baseline pipeline it is composed with.

The improvement transfers across base-model families. Switching to Qwen3-8B and the SWE-Lego post-training pipeline, mid-training improves SWE-Bench-Verified by $+3.20$ points and SWE-Bench-Lite by $+5.40$ points, comparable to the gains observed on Qwen2.5-Coder. We note that this comparison varies the post-training pipeline simultaneously with the base model (SWE-Lego is required because Qwen3-8B's $40{,}960$-token context exceeds that of the R2E-Gym/SWE-Smith scaffolds), so the cross-base-model result should be read as "the recipe is not specific to the Qwen2.5-Coder family under one alternative agentic pipeline," rather than as a guarantee that it generalizes across all base-model families. We elaborate this caveat in Section 6. The full Qwen2.5-Coder scaling curves (7B/14B/32B) are given in Appendix C.

Table 2: Capability preservation and cross-domain transfer at 14B with the R2E-Gym post-training pipeline. "Non-agent coding" tests pure code generation; "Agent OOD" is closer to coding agents but in a different environment; "Tool use" tests non-coding tool-call behavior. Bold marks the better trained variant per column (post-only vs. ours); the Instruct row is shown as a reference ceiling and is not in competition. All cells are percentages.

	Non-agent coding			Agent OOD	Tool use		
Setting	LiveCode	OJBench	FSB-EN	Terminal	τ-bench	BFCL	Avg
Instruct Model	37.20	5.20	53.80	0.00	5.70	23.20	20.85
+ R2E-Gym	24.10	2.80	47.72	2.41	3.40	15.80	16.04
+ FIM Mid-Train + R2E-Gym	**35.20**	**4.74**	**48.25**	**3.66**	**7.30**	**18.20**	**19.56**
Δ (vs. R2E-Gym only)	$+11.10$	$+1.94$	$+0.53$	$+1.25$	$+3.90$	$+2.40$	$+3.52$

3.3 Capability Preservation and Cross-Domain Transfer

A natural concern with any post-training stage that specializes a model toward a narrow agent task is that it may erode capabilities the base model already possessed. We test for this directly on six benchmarks that sit outside the SWE-Bench distribution and, in the case of τ-bench and BFCL, outside the coding domain entirely. Because this analysis requires a controlled comparison among (instruct base) vs. (post-training only) vs. (mid-training + post-training), and the corresponding controlled triple is the most expensive to produce, we conduct it at the 14B scale with the R2E-Gym pipeline. Results are summarized in Table 2.

Agentic post-training has a substantial hidden capability cost. Applying R2E-Gym alone to the 14B base reduces every non-agent coding and tool-use benchmark relative to the Instruct ceiling: LiveCodeBench drops by 13.10 points, BFCL by 7.50, FullStackBench-EN by 6.08, τ-bench by 5.30, and OJBench by 2.40. Averaged across the six benchmarks the post-trained model loses 5.52 points relative to the Instruct ceiling. This regression is rarely highlighted in agent papers, but it is the implicit cost being paid for SWE-Bench gains.

Mid-training largely closes the regression gap. Adding FIM mid-training before the same post-training restores LiveCodeBench by $+11.10$, OJBench by $+1.94$ (within 0.46 of the Instruct ceiling), and FullStackBench-EN by $+0.35$. The six-benchmark average rises from 15.33 to 18.85, an increase of $+3.52$ over the post-training-only configuration, while the SWE-Bench-Verified gain on the same base is preserved (Table 1). In other words, mid-training improves the cost–benefit profile of agentic post-training: the in-domain target metric improves, and the bulk of the off-distribution capability erosion is undone in the same training run.

Cross-domain transfer to non-coding tool use. The most direct test of our motivating hypothesis lies in the rightmost columns of Table 2: τ-bench and BFCL contain no Python code-editing data, and our mid-training corpus contains no tool-use trajectories. Mid-training nevertheless improves τ-bench by $+4.00$ and BFCL by $+2.50$ over post-training alone, with a directionally consistent recovery on the also-OOD Terminal-Bench ($+1.19$). Because the mid-training data carries no tool-use signal a priori, the only mechanism by which it can affect these benchmarks is by installing a structural prior at mid-training time that survives subsequent post-training. The fact that this prior moves τ-bench and BFCL in the same direction as the in-domain SWE-Bench gains is consistent with our hypothesis that the function-call site and the agent tool-call step share an underlying conditioning structure (Section 2.1).

3.4 Ablation Studies

We conduct two controlled ablations, both on the Qwen2.5-Coder-7B base with R2E-Gym post-training and a shared mid-training-free baseline. The first isolates the role of the chain-of-thought rationale embedded in the FIM middle span; the second isolates the role of the function-aware selection pipeline. All selection variants in Block (B) operate under a controlled budget: each variant selects exactly 100K single functions for masking, and basic hard filters (LoC $\in [10, 200]$, dunder methods excluded) are applied uniformly. Differences across rows therefore reflect *which* functions are selected rather than *how many*. Mask granularity beyond single functions, score-threshold

Table 3: Ablation studies on the 7B model with R2E-Gym post-training. **(A)** varies the rationale source in the FIM middle. **(B)** varies the function-selection algorithm. **(C)** varies mask granularity by mixing in multi-function groups; (A) and (B) use single functions only. All blocks share the baseline (*w/o mid-train*) and a controlled 200K-target budget; CoT is fixed to Gemini-3-Flash in (B) and (C). Bold marks the best configuration in each block; the 85%/10%/5% mixture in (C) is the recipe used in our main results (Table 1), which is trained on the full corpus rather than the 200K budget here.

Setting	SWE-Bench-Verified	SWE-Bench-Lite	Average
(A) Chain-of-thought source (selection fixed to "full")			
w/o mid-train (baseline)	15.00	11.33	13.17
+ FIM, no CoT	15.60	11.00	13.30
+ FIM, self-CoT (Qwen2.5-Coder-7B)	16.40	13.30	14.85
+ FIM, Gemini-3-Flash CoT	**17.00**	**14.20**	**15.60**
(B) Function-selection algorithm (CoT fixed to Gemini-3-Flash)			
w/o mid-train (baseline)	15.00	11.33	13.17
Random	15.30	12.60	13.95
Gemini-selected	16.40	13.70	15.05
PDG only	16.10	13.60	14.85
PDG + complexity ($\hat{H}$)	16.50	13.60	15.05
PDG + inferability ($\hat{I}$)	16.70	14.00	15.35
Full (PDG + $\hat{H}$ + $\hat{I}$)	**17.00**	**14.20**	**15.60**
(C) Mask granularity (selection fixed to "full", CoT fixed to Gemini-3-Flash)			
w/o mid-train (baseline)	15.00	11.33	13.17
Single-function only	17.00	14.20	15.60
90% single + 10% pair ($k = 2$)	17.20	14.60	15.90
95% single + 5% triple ($k = 3$)	17.00	14.40	15.70
85% single + 10% pair + 5% triple	**17.40**	**14.80**	**16.10**

sensitivity, and mid-training token budget are reserved for future investigation; due to compute constraints, the ablations run at the 7B scale only.

FIM structure contributes independently of CoT distillation. Block (A) of Table 3 disarms a natural reviewer concern: that our improvements stem primarily from distilling reasoning from a frontier model. Removing the rationale entirely (*no CoT*) still improves the average by $+0.13$ over the no-mid-training baseline, with a $+0.6$ gain on SWE-Bench-Verified, indicating that the function-aware FIM objective by itself contributes some signal even without any external rationale. Replacing Gemini-3 with rationales generated by the model being trained (*self-CoT*) reaches an average of 14.85, recovering 1.7 of the 3.2-point gap between *no CoT* and *Gemini-3 CoT*; high-quality external rationales help, but a sizable fraction of the gain is reproducible without a frontier teacher.

The function-selection algorithm is the dominant lever within mid-training. Block (B) varies the selection algorithm with the CoT source held fixed to Gemini-3 and the selection budget fixed to 100K single functions, so that all rows incur identical mid-training cost. *Random* masking establishes the floor: with no structural priors beyond the basic hard filters, mid-training reaches XX.XX on average, a margin of +X.XX over no mid-training. Letting Gemini-3 directly choose which functions to mask (*Gemini-selected*)—a strong non-structural baseline that uses the same teacher we use for rationales—reaches XX.XX, indicating that frontier judgment on *which* function to mask is helpful but not by itself sufficient. Restricting the candidate pool to functions with at least one neighbor in the program dependency graph (*PDG only*) reaches XX.XX, a further +X.XX over Random: functions embedded in caller-callee or sibling structure are intrinsically more useful as FIM targets than isolated ones, even before any scoring is applied. Adding a scoring function on top of the PDG pool yields the next layer of gain: *PDG + complexity* reaches XX.XX and *PDG + inferability* reaches XX.XX, showing that both the intrinsic difficulty of the target and its recoverability from surrounding context independently contribute beyond the structural filter. Combining the two scores via the harmonic-mean-like $\hat{H}\,\hat{I}/(\hat{H} + \hat{I})$ form—the *full* configuration—reaches 16.40, a further +X.XX over the better single-score variant. The combination is therefore not redundant: $\hat{H}$ and $\hat{I}$

measure complementary properties of a candidate, and selecting targets that score highly on only one of the two yields either unlearnable noise (high complexity, low inferability) or trivial fills (low complexity, high inferability).

4 Analysis

The previous section established that FIM mid-training improves end-task metrics. We now ask *how* the resulting agent behaves differently and *where* along the trajectory the gains accrue, focusing on the two findings that most directly bear on the motivating hypothesis. Both analyses are performed on the 14B configuration with R2E-Gym post-training, comparing the post-training-only baseline (*R2E-Gym*) against our recipe (*FIM-Midtrain + R2E-Gym*). Every trajectory-level statistic is computed over three independent evaluation runs of each checkpoint and reported as the run-mean on the SWE-Bench-Verified test set (500 instances per run); the corresponding analysis on SWE-Bench-Lite, the full action-type distribution, the no-patch mechanism, and a concrete trajectory contrast are deferred to Appendix D and **??**.

4.1 Recovery from Negative Observations

We define a trajectory as containing a *negative observation* if any step's tool output matches one of a fixed set of error patterns (stack traces, "`No replacement was performed`", "`SyntaxError`", shell errors, etc.; full list in Appendix B). The fraction of such trajectories is essentially identical between the two checkpoints (88.8% baseline vs. 91.8% ours), so the agents are exposed to comparable amounts of negative feedback. The *recovery rate*—the fraction of error-containing trajectories that nevertheless terminate with a passing patch—rises from 24.8% to 28.8% (+4.0 pp, a 16% relative increase; Table 4). This metric tracks the precise capability our framing predicts mid-training should support: continuing productively after an external return contradicts the model's prior expectation. Mid-training also shifts the agent toward an iterate-and-verify policy, increasing edits per solved task from 3.3 to 7.4 and trajectory length from 15.1 to 23.6 steps; detailed action-type breakdowns are in Appendix D.

Table 4: Headline trajectory-level metrics on SWE-Bench-Verified (14B, R2E-Gym pipeline), averaged over three independent evaluation runs of each checkpoint. "Recovery rate" is the fraction of trajectories containing a negative observation that terminate with a passing patch; "Edits / solved" is the mean number of `file_editor` operations on solved tasks. Full metrics, including unsolved-trajectory statistics, action-type distributions, and per-checkpoint output/context summaries, are in Appendix D.

Setting	Recovery rate (%)	Edits / solved	Steps / solved	Pass (%)
+ R2E-Gym	24.8	3.3	15.1	26.2
+ FIM-Midtrain + R2E-Gym	**28.8**	**7.4**	**23.6**	**29.2**

4.2 The Gain Concentrates on Multi-Function Reasoning

The most direct test of our isomorphism argument (Section 2.1) is whether mid-training preferentially helps tasks whose successful resolution requires reasoning about cross-function dependencies inside a file. We stratify the 500 Verified tasks by the shape of the gold patch (Figure 3). On the 88 tasks whose gold patch modifies ≥ 2 functions within a single file, the baseline solves 13.6% on average and ours solves 22.7%, an absolute gain of +9.1 pp—more than $4\times$ the gain on the 341 single-function tasks (+2.1 pp). Per-instance head-to-head on this bucket shows ours uniquely solves about twice as many tasks as the baseline does. Multi-file tasks (n=71) are not differentially helped by mid-training, which we attribute to a granularity mismatch (our FIM operates within files); discussion is deferred to Appendix D.

We read this concentration as a closing-loop confirmation of the isomorphism argument in Section 2.1: the slice where mid-training helps most is precisely the slice where the agent must follow control- and data-dependencies between caller-callee or sibling functions inside a file—the same structure exposed by function-aware FIM masking. A complementary outcome-distribution breakdown (Figure 7 in Appendix) shows that the +15-task gain is driven by an order-of-magnitude reduction in *no-patch*

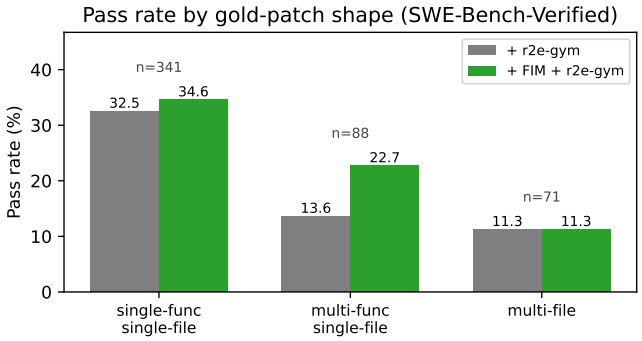

Figure 3: Pass rate on SWE-Bench-Verified (14B + R2E-Gym) stratified by the shape of the gold patch, averaged over three evaluation runs per checkpoint. The largest absolute gain is on *multi-function single-file* tasks ($+9.1$ pp), more than $4\times$ the gain on single-function tasks ($+2.1$ pp); multi-file tasks remain hard for both checkpoints ($\sim 11.3\%$ each).

failures alongside small reductions in localization errors; we interpret the no-patch collapse as a direct consequence of the FIM training signal, which conditions the model to produce a non-empty span between the prefix and suffix—a disposition that survives subsequent post-training.

5 Related Work

Mid-training and continued pretraining. A growing line of work has identified a distinct training stage between large-scale pretraining and task-specific post-training, in which a model is further trained on a smaller but more carefully curated corpus to install inductive biases that are difficult to acquire from either generic web text or downstream fine-tuning data. MiniCPM [6] and OLMo [4] explicitly schedule such a stage to upweight high-quality or domain-relevant data near the end of pretraining, and Code-Llama [16] long-context adaptation can be read as a mid-training stage targeting context-length generalisation. We adopt the same staging philosophy but apply it to a different end: rather than optimising for general data quality or context length, we use mid-training as the natural place to install an *agent-oriented* structural prior through function-aware FIM. Positioning the objective at this stage concentrates its signal immediately before agentic post-training rather than diluting it across trillions of pretraining tokens.

Fill-in-the-middle objectives. The fill-in-the-middle objective [1] has become a standard ingredient in modern code LLM pretraining, with random-span variants mixed into the pretraining corpora of Code-Llama [16], StarCoder [11], Qwen-Coder [7], and DeepSeek-Coder [5]. Our work builds directly on this body of evidence that bidirectional infilling is a useful complement to left-to-right modelling, and we explicitly view our recipe as an *extension and specialisation* of prior FIM rather than a replacement. Three differences mark our setting. First, masking targets are chosen at *function granularity* via program dependency graph analysis and a complexity–inferability double criterion, so that the masked region corresponds to a unit of reasoning rather than a syntactically arbitrary span. Second, we embed an *explicit chain-of-thought* inside the FIM middle so that the model is supervised to produce a rationale before the code, mirroring the think-then-act structure of an agent step. Third, the objective is applied in a *dedicated mid-training stage* rather than dissolved into pretraining, so that the structural prior reaches post-training in concentrated form. None of the three differences is in tension with prior random-span FIM; together, they specialise the same family of objectives to coding-agent capability.

Coding agent foundation models. Recent progress on coding agents has been driven by benchmarks such as SWE-Bench [10], scaffolds such as SWE-agent [21] and OpenHands [18], and trajectory-centric post-training pipelines such as SWE-Gym [14], r2e-gym [9], and SWE-Smith [22]. These pipelines synthesise or curate agent trajectories that imitate human or LLM behaviour on issue-resolution tasks, and they have repeatedly demonstrated that high-quality trajectory data is sufficient to push end-task numbers. Our work is complementary rather than competing: the post-training

pipelines we use in our main results are r2e-gym and SWE-Smith, applied without modification on top of a mid-trained checkpoint. The novelty lies one stage earlier. Instead of producing more synthetic trajectories, we extract a self-supervised signal from the structure already present in ordinary source code, and we show that this signal both raises in-domain agent metrics and mitigates the off-domain capability erosion that trajectory-only post-training otherwise inflicts.

Distillation from frontier models. Our default recipe uses Gemini-3-Preview to generate the chain-of-thought rationales embedded inside each FIM middle span, placing it within the broader tradition of distilling capability from a strong teacher into a smaller student [13, 19, 3]. We acknowledge this dependency upfront because it is essential for an honest reading of our results. The role of the rationale here, however, is not to transfer the teacher's generic reasoning ability but to provide a structured intermediate supervision aligned with the FIM objective. We isolate the contribution of FIM structure from that of CoT distillation through a controlled ablation in Section 3.4: removing the rationale entirely still beats the no-mid-training baseline, and replacing the frontier teacher with self-generated rationales recovers a sizeable fraction of the gain. We therefore do not view the recipe as fundamentally distillation-driven, even though the best configuration does benefit from teacher rationales.

6 Limitations and Discussion

We close by stating six limitations of the present study, both to scope our claims and to flag concrete directions for follow-up work.

Limitation 1: Python-centric evaluation. The mid-training corpus is exclusively Python, and the in-domain agent benchmarks (SWE-Bench-Verified and SWE-Bench-Lite) are likewise Python-centric. Cross-language generalisation is supported only indirectly, through the FullStackBench-ZH bilingual coding benchmark in Section 3.3, and we do not cover Java, C++, Rust, or other language ecosystems in the main tables. Whether the function-aware FIM recipe transfers to languages with different module systems, type disciplines, or call-resolution semantics is left to future work.

Limitation 2: Dependency on a frontier teacher for CoT. Our default recipe relies on the Gemini-3-Preview API to synthesise the rationale embedded inside each FIM middle span. The CoT-source ablation in Section 3.4 shows that self-generated rationales recover most of the gain and FIM structure alone already beats the no-mid-training baseline, so the recipe is not fundamentally distillation-bound; nonetheless, a fully open-source replication of our best configuration requires either a comparably strong open teacher or acceptance of the residual gap reported in that ablation.

Limitation 3: Imperfect program dependency graph analysis on dynamic Python. Our PDG construction operates on the abstract syntax tree and resolves calls via a combination of qualified-name matching and short-name fallback. This works well for the vast majority of canonical Python code but is imperfect on idioms involving dynamic dispatch, decorator chains, and monkey patching, where the static call graph and the run-time call graph diverge. We characterise these failure modes and the corresponding mitigations in Appendix B; an analysis-aware extension that incorporates light dynamic information is a natural next step.

Limitation 4: Cross-base-family validation is partial. The cross-base-model evidence in Section **??** comes from a single non-Qwen2.5-Coder configuration (Qwen3-8B paired with the swe-lego post-training pipeline). Because moving from Qwen2.5-Coder to Qwen3-8B simultaneously varies the post-training pipeline (swe-lego is required to fit Qwen3's $40{,}960$-token context), the cross-family result should be read as "the recipe is not specific to a single pretraining/post-training combination" rather than as a guarantee that it generalises across all base-model families. Validations on Llama, the DeepSeek family, and other architectures are not covered here.

Limitation 5: Capability-preservation comparison only at 14B. The controlled three-row comparison in Section 3.3 (instruct base vs. post-training only vs. mid-training plus post-training) is conducted at the 14B scale only, due to the cost of running the full set of non-agent coding and tool-use benchmarks at every scale. The 7B and 32B in-domain SWE-Bench trends are consistent with the 14B picture and suggest a similar regression-recovery pattern, but we do not run the fully

controlled triple at those scales and therefore do not make a quantitative claim about capability preservation outside 14B.

Limitation 6: Modularity assumption of function-aware selection. Function-aware FIM presupposes that the underlying code is meaningfully modular, so that masking a single function or a small connected group produces a non-trivial reasoning target. On extreme monolithic code or inline-heavy codebases—scripts dominated by top-level statements, generated code with few callable abstractions, or notebooks—the selection pipeline has fewer eligible targets and may degrade gracefully to a much smaller usable subset of files. We do not study this regime systematically here; both characterising the breakdown point and extending the algorithm with code-block-level abstractions for non-modular code are open directions.

7 Conclusion

We started from the observation that a single step of a coding agent and a single function call site share the same four-part conditioning structure—*context, action, externally produced return, continuation*—and that this structural isomorphism makes ordinary source code a natural, internet-scale supply of agent-relevant signal. Building on this framing, we introduced function-aware FIM mid-training: a self-supervised stage that masks targets selected through program dependency graph analysis and a complexity–inferability double criterion, with chain-of-thought rationales embedded inside the FIM middle span. Across three independent axes of robustness—model size (7B/14B/32B), post-training pipeline (r2e-gym, SWE-Smith, swe-lego), and base-model family (Qwen2.5-Coder, Qwen3)—the recipe delivers consistent gains on coding-agent benchmarks, and the same Python-only mid-training corpus also transfers to non-coding tool-use benchmarks (τ-bench, BFCL) where it has no a priori reason to help, providing direct evidence for the motivating isomorphism. Promising directions for future work include extending the function-aware selection pipeline to non-Python languages, validating the mid-training stage on additional base-model families beyond the Qwen lineage, and studying how mid-training composes with reinforcement-learning-based post-training, where a structural prior over function-call-shaped trajectories may shape the exploration landscape in ways that supervised post-training alone cannot.

References

[1] Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle, 2022.

[2] ByteDance Seed Foundation Code Team. FullStack Bench: Evaluating LLMs as full stack coders, 2024.

[3] Sanchit Gandhi, Patrick von Platen, and Alexander M. Rush. Distil-Whisper: Robust knowledge distillation via large-scale pseudo labelling, 2023.

[4] Dirk Groeneveld, Iz Beltagy, Pete Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Harsh Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, Shane Arora, David Atkinson, Russell Authur, Khyathi Raghavi Chandu, Arman Cohan, Jennifer Dumas, Yanai Elazar, Yuling Gu, Jack Hessel, Tushar Khot, William Merrill, Jacob Morrison, Niklas Muennighoff, Aakanksha Naik, Crystal Nam, Matthew E. Peters, Valentina Pyatkin, Abhilasha Ravichander, Dustin Schwenk, Saurabh Shah, Will Smith, Emma Strubell, Nishant Subramani, Mitchell Wortsman, Pradeep Dasigi, Nathan Lambert, Kyle Richardson, Luke Zettlemoyer, Jesse Dodge, Kyle Lo, Luca Soldaini, Noah A. Smith, and Hannaneh Hajishirzi. OLMo: Accelerating the science of language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[5] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence, 2024.

[6] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia,

Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. MiniCPM: Unveiling the potential of small language models with scalable training strategies, 2024.

[7] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-Coder technical report, 2024.

[8] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code, 2024.

[9] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents, 2025.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[11] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder: May the source be with you!, 2023.

[12] Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026.

[13] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of GPT-4, 2023.

[14] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2024.

[15] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[16] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code Llama: Open foundation models for code, 2023.

[17] SWE-Lego Team. SWE-Lego: Pushing the limits of supervised fine-tuning for software issue resolving, 2026.

[18] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Yuan, Hao Pan, Yueqi Yang, Jiayi Hu, Mingyang Tan, Yongliang Liu, Yujia Liu, Tianbao Du, Hao Zhang, Yan Zhang, Jiawei Yu, Wanjun Zhao, Robert M. Yang, Jingbo Yi, Mingzhang Zhang, Yuxiang Wang, Jiawei Wei, Theo Olausson, Tianyu Liu, Ren-Biao Sun, Wendong Liu, Tianyang Yang,

Frank Mao, Yu Chen Wang, Wei Liu, Bill Lin, Boyuan Wei, Hanzhe Liang, Yifan Yan, Yibo Sun, Yuhan Sun, Hangrui Liu, Heran Wang, Hao Yan, Tao Tang, Yufeng Yang, Vijay Reddy, Saatvik Gandhi, Pravir Singh, Yu Yu, Tianhao Liu, Jianan Pang, Yumeng Wang, Saurabh Geng, Yang Hu, Lily Zhao, Yipeng Zhao, Pinjia Liu, Junda Bian, Junyi Mu, Junyu Tan, Hao Wang, Wangchunshu Lai, Shivansh Garg, Yunzhu Wang, Robert Lo, Junda Tao, Junyu Liu, Yifei Lyu, Yueting Wang, Mingyu Hu, Boqin Yang, Bo Liu, Hao Sun, Yangruibo Wang, John Yang, Mohit Bansal, Rada Mihalcea, Yang Cao, Edward Liu, Tongshuang Du, Ofir Press, Percy Liang, Justin Heinz, Vincent J. Hellendoorn, Marcel Boehme, Lianmin Zheng, Ion Stoica, Junfeng Wang, Peter Henderson, Prateek Goyal, Eric Mitchell, Heng Ji, Charlie Frank, Dan Hendrycks, Yiming Liang, Maosong Sun, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[19] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[20] Zhexu Wang, Yiwen Yang, Pengfei Zhu, Chuanrui Zheng, Jianxin Hu, Wangyue Huang, Yuchen Wu, Weizhe Liu, Shilong Wang, Yixuan Yan, Yiwen Liu, Yufei Sui, Lewei Zhang, Tianyu Yang, Mingyang Yu, Junjie Zhao, Weiqian Zhuang, Ke Lu, Junjie Yang, Cong He, Jian-Guang Lou, Zhouhan Lin, Junran Liu, Yidan Tao, and Tianhe Hua. OJBench: A competition level code benchmark for large language models, 2025.

[21] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent–computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[22] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents, 2025.

[23] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

A Corpus Details

This appendix expands on the data collection summary in Section 2.2. The full repository list, license inventory, and commit hashes will be released; here we report category coverage and per-property statistics.

B Algorithmic Details

This appendix collects the full algorithmic details that are summarized in Section 2.3.

B.1 Single-Function FIM Target Selection

Algorithm 1 summarizes the per-file pipeline that produces single-function FIM targets. It composes the program dependency graph (Section 2.3.1), the complexity score $\hat{H}$ (Section 2.3.2), the inferability score $\hat{I}$ (Section 2.3.3), the difficulty penalty ρ (Appendix B.5), and hard filters described below.

B.2 Program Dependency Graph: Call Resolution

For every `Call` node inside the body of $u \in \mathcal{V}$, we attempt to resolve its callee to some $v \in \mathcal{V}$, adding a directed edge $u \to v$ in $\mathcal{E}_{\text{call}}$. Resolution handles three Python idioms:

- **Direct module-level invocations**: `foo(...)` is resolved by qualified-name lookup in the same module.

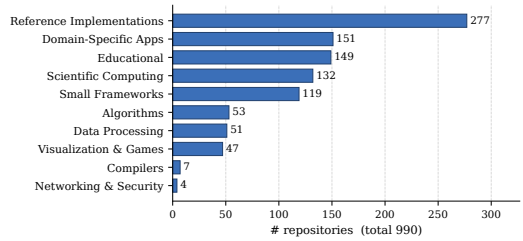

Figure 4: Distribution of the 990 source repositories across ten topic categories. The corpus is dominated by reference implementations, scientific computing, and small frameworks; compilers and networking/security tails remain by design to maintain coverage diversity.

Figure 5: Mid-training corpus statistics after decontamination and quality filtering. Token counts are reported with the Qwen2.5-Coder tokenizer.

Property	Value
Source GitHub repositories	990
Topic categories	10
Filtered self-contained Python files	$\approx$ 78K
Total lines of Python code	$\approx$ 33M
Mean LoC per file	$\approx$ 425
Mid-training token budget	$\approx$ 3B
Single-function FIM targets	$\approx$ 412K
Multi-function targets (k=2)	$\approx$ 96K
Multi-function targets (k=3)	$\approx$ 31K
Mean target LoC	$\approx$ 38
Targets with Gemini-3 CoT	100%
SWE-Bench source-repo overlap	0
Commit cutoff	before earliest SWE-Bench base
License inventory	permissive only

Algorithm 1 Single-function FIM target selection for one file.

Require: Source file s; thresholds $\tau_d, \tau_{\text{FIM}}, \tau_{\hat{H}}$

$T \leftarrow \text{ParseAST}(s)$

$\mathcal{V}, \mathcal{E}_{\text{call}}, \mathcal{E}_{\text{sib}} \leftarrow \text{BuildPDG}(T)$ ▷ Section 2.3.1

$\mathcal{C} \leftarrow \emptyset$

for each $v \in \mathcal{V}$ **do**

  if v is a dunder method **or** $\text{LoC}(v) \notin [10, 200]$ **then**

    continue

  end if

  Compute $\hat{H}(v)$ and $\hat{I}(v)$ ▷ Eqs. (1), (2)

  if $\hat{H}(v) < \tau_{\hat{H}}$ **then continue**

  end if

  $\Delta(v) \leftarrow \max(0, \hat{H}(v) - \hat{I}(v))/(\hat{H}(v) + \epsilon)$

  $\text{FIM}(v) \leftarrow \dfrac{\hat{H}(v)\,\hat{I}(v)}{\hat{H}(v) + \hat{I}(v) + \epsilon} \cdot \rho(\Delta(v))$

  if $\text{FIM}(v) \geq \tau_{\text{FIM}}$ **then**

    $\mathcal{C} \leftarrow \mathcal{C} \cup \{v\}$

  end if

end for

return $\mathcal{C}$ ▷ ranked by FIM descending

- **Class instantiation**: `ClassName(...)` is resolved to `ClassName.__init__`.
- **Method calls**: `self.m(...)` or `cls.m(...)` inside class bodies is resolved within the enclosing class scope.

When a call cannot be resolved exactly we fall back to short-name matching against the qualified-name index, which recovers most cross-module references while admitting a controlled false-positive rate. Sibling edges $\mathcal{E}_{\text{sib}}$ are constructed by enumerating all pairs of methods belonging to the same class.

B.3 Complexity Score: Caps and Weights

The complexity score $\hat{H}(v)$ defined in Eq. (1) uses caps $(c_\ell, c_c, c_d) = (50, 10, 5)$ and weights $(w_\ell, w_c, w_d) = (0.4, 0.4, 0.2)$. The cap $c_\ell = 50$ on lines of code matches the median target length in our corpus; the cap $c_c = 10$ on McCabe cyclomatic complexity is roughly twice the median over our filtered functions; the cap $c_d = 5$ on nesting depth covers the tail of practical Python code. The cap value of 2 in $\phi(x, c) = \min(x/c, 2)$ allows genuinely complex functions to score above the median without letting outliers dominate.

B.4 Inferability Score: Component Formulas

The five components of the inferability score $\hat{I}(v)$ (Eq. 2) are defined as follows. Weights are $(\alpha, \beta, \gamma, \delta, \varepsilon) = (0.30, 0.25, 0.20, 0.10, 0.15)$.

Caller signal C_{caller}. For each caller u of v, we score the specificity of its call sites by inspecting the syntactic form of each argument: literal constants are maximally informative, named variables less so, keyword arguments contribute proportionally, and a base score is awarded for the call site itself. The sum is normalized by an expected-callers constant.

Callee signal C_{callee}. The number of intra-file functions that v itself calls, normalized by a small expected-fan-out constant. A function that orchestrates known helpers is more recoverable than one that calls only opaque externals.

Signature signal C_{sig}. A monotone aggregate of return-type annotation, parameter-type annotations, the descriptiveness of the function name (decomposed into snake-case parts), and the count of non-`self` parameters.

Documentation signal C_{doc}. Indicator of docstring presence. We do not attempt to score docstring informativeness, both for robustness and because Section 2.4 generates a richer textual rationale.

Class signal C_{class}. For methods, the number of in-class siblings (normalized) plus a small bonus when an `__init__` is present in the same class.

B.5 Difficulty Δ, Penalty ρ, and Hard Filters

The penalty $\rho(\Delta(v))$ in Eq. (3) is built from the residual difficulty $\Delta(v)$, defined as the share of complexity that is not explained by context:

$$\Delta(v) \;=\; \frac{\max\Big(0,\, \hat{H}(v) - \hat{I}(v)\Big)}{\hat{H}(v) + \epsilon} \;\in\; [0, 1). \tag{4}$$

The penalty ρ is one-sided: targets with $\Delta(v) \le \tau_d$ are left intact, while targets above the threshold are damped by a Gaussian factor:

$$\rho(\Delta) \;=\; \begin{cases} 1, & \Delta \le \tau_d, \\ \exp\!\left(-\dfrac{(\Delta - \tau_d)^2}{2\sigma^2}\right), & \Delta > \tau_d. \end{cases} \tag{5}$$

We use $\tau_d = 0.5$ and $\sigma = 0.2$. The asymmetry reflects an asymmetry in the underlying objective: trivially easy targets are eliminated by the hard filters below, so we do not need to additionally reward high inferability; conversely, targets that remain hard even with full context become unlearnable noise and must be down-weighted.

Hard filters. Independent of the smooth score, we discard *(i)* functions with $\mathrm{LoC}(v) \notin [10, 200]$ to control training-instance length; *(ii)* dunder methods (`__init__`, `__repr__`, etc.), which serve as scaffolding rather than reasoning targets; *(iii)* functions with $\hat{H}(v) < 0.15$ to remove trivial cases; and *(iv)* functions with $\mathrm{FIM}(v) < 0.08$. A file with no surviving target contributes nothing to the corpus, ensuring each training sample carries informative signal.

B.6 Multi-Function Group Score

Group score. For a group $G = \{v_1, \ldots, v_k\}$ enumerated from one of the eight topology patterns (listed below), we define

$$\mathrm{FIM}(G) \;=\; \underbrace{\mathrm{Coup}(G)}_{\text{coupling}} \cdot \frac{\hat{H}(G)\,\hat{I}(G)}{\hat{H}(G) + \hat{I}(G) + \epsilon} \;\cdot\; \rho(\Delta(G))\,, \tag{6}$$

where $\hat{H}(G) = \frac{1}{k}\sum_v \hat{H}(v)$ averages individual complexities, $\Delta(G)$ is defined analogously to the single-function case, and the coupling term $\mathrm{Coup}(G) \in [0, 1]$ is a normalized count of intra-group edges plus a shared-state ratio:

$$\mathrm{Coup}(G) \;=\; w_{\mathrm{c}}\,\frac{|E_G^{\mathrm{call}}|}{k(k-1)} + w_{\mathrm{s}}\,\frac{|E_G^{\mathrm{sib}}|}{\binom{k}{2}} + w_{\mathrm{st}}\,\overline{\mathrm{Jacc}}_G, \tag{7}$$

with $\overline{\mathrm{Jacc}}_G$ the mean Jaccard similarity over pairs in G between sets of accessed instance attributes (`self.x`). We use $(w_{\mathrm{c}}, w_{\mathrm{s}}, w_{\mathrm{st}}) = (0.50, 0.20, 0.30)$. The coupling term enters multiplicatively because, unlike the single-function case, the value of a multi-function target derives specifically from cross-function dependencies; a weakly coupled group is indistinguishable from k independent samples.

Group inferability. $\hat{I}(G)$ is recomputed under the assumption that *all* members of G are masked simultaneously: caller, callee, and sibling contributions from inside G are excluded, and docstrings of group members contribute zero (they are part of the masked body). Signature information is retained because we do not mask function signatures. This re-computation prevents groups from receiving spurious credit for intra-group references that disappear under joint masking.

Topology taxonomy. A candidate group must form a connected subgraph in $\mathcal{E}_{\mathrm{call}} \cup \mathcal{E}_{\mathrm{sib}}$. We enumerate eight patterns:

- $k = 2$: *caller-callee* (direct $A \to B$ edge); *co-callee* (two callees of a common caller); *sibling-coupled* (same-class methods sharing ≥ 1 instance attribute); *mutual-call* ($A \rightleftharpoons B$).
- $k = 3$: *call-chain* ($A \to B \to C$); *hub* ($A \to B,\ A \to C$); *fan-in* ($B \to A,\ C \to A$); *class-triad* (three pairwise state-sharing methods of the same class).

Group selection. We score every candidate group emitted by the topology enumerator and discard those with insufficient coupling, complexity, or score (defaults: $\mathrm{Coup}(G) \geq 0.20$; $\hat{H}(G) \geq 0.18$; $\mathrm{FIM}(G) \geq 0.10$). Groups exceeding LoC ratios of 0.30 ($k = 2$) or 0.40 ($k = 3$) of the file are rejected. The remaining candidates are sorted by $\mathrm{FIM}(G)$ and selected greedily under non-overlap: a function may belong to at most one selected group per file. Per-file caps (5 pairs, 3 triples) prevent any single file from dominating the corpus.

B.7 Chain-of-Thought Prompt Template

For each selected target, the chain-of-thought rationale embedded inside the FIM middle span (Section 2.4) is generated by Gemini-3-Preview using the following prompt skeleton:

> *[System]* You are an expert software engineer.
> *[User]* Below is a Python source file with one function body redacted (shown as `<MASKED>`).
>
> `<file with target body redacted>`
>
> The redacted function has the following signature and (if present) docstring:
>
> `<signature + docstring>`
>
> The original function body was:
>
> `<original body>`

Write a concise step-by-step rationale (in natural language) that explains how the body's behavior could be **recovered** from the surrounding file—for example, by referencing call sites, helper functions, and class-level state—without simply restating the code.

The rationales are constrained to be at most 300 tokens and are filtered by length and basic format checks before training. We use greedy decoding; details on retry logic and quality filters are at <repoURLplaceholder>.

C Consistency Across Model Sizes

This appendix expands on the cross-scale consistency claim made in Section 3.2, replotting the in-domain numbers from Table 1.

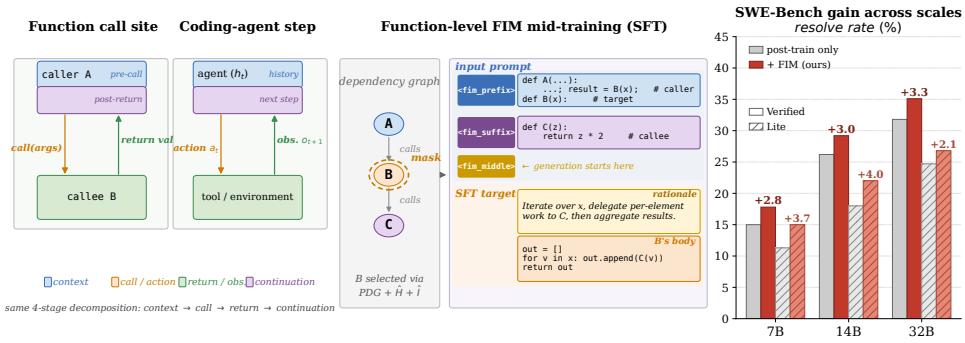

Figure 6: SWE-Bench-Verified (left) and SWE-Bench-Lite (right) for the Qwen2.5-Coder family at 7B, 14B, and 32B, comparing post-training only (grey) against FIM mid-training followed by the same R2E-Gym post-training pipeline (red). Annotations report the absolute gain from mid-training at each scale.

The gain does not vanish with scale. A natural concern with any structural prior is that larger pretrained models, having seen more code tokens, would already have absorbed it; in that case mid-training would help only at small scale and diminish toward 32B. The data we have do not show this pattern: the absolute SWE-Bench-Verified gain is $+2.80$ at 7B, $+3.00$ at 14B and $+3.30$ at 32B, a narrow band of variation as the parameter count grows $4\times$. The same direction holds on SWE-Bench-Lite ($+3.67/+4.00/+2.10$). The relative gain naturally contracts with size ($+18.7\%$ at 7B, $+11.5\%$ at 14B, $+10.4\%$ at 32B), consistent with the higher absolute baseline at larger scales. We do not extrapolate beyond the three sizes evaluated; whether the trend continues at 70B+ is left to future work.

D Extended Behavioral Analysis

This appendix collects behavioral and failure-mode results that supplement the core analysis in Section 4. Unless stated otherwise, all statistics are computed from three independent evaluation runs of each checkpoint on SWE-Bench-Verified (500 instances per run) and reported as run-means.

D.1 Full Trajectory Metrics

Table 5 reports the full trajectory metrics summarized in Table 4, including unsolved-task breakdowns and output/context summaries.

D.2 Iterate-and-Verify: Action-Type Distribution

Mid-training shifts the action-type distribution toward an iterate-and-verify policy. The share of `search` actions falls from 15.3% to 11.0%, while the share of `execute_bash` actions rises from 19.5% to 24.6%: the agent spends a smaller fraction of its budget on passive repository lookup and a larger fraction on actively running scripts and tests, then refining its edit. The cost is more steps—ours

Table 5: Trajectory-level behavioral metrics on SWE-Bench-Verified (14B, R2E-Gym pipeline), averaged over three independent evaluation runs of each checkpoint. Steps, edits, and errors-encountered are means per task. "Empty patch" is the fraction of *failed* trajectories whose final `output_patch` is empty; "Loc. correct" is the fraction of all trajectories that locate at least one file overlapping the gold patch; "Ctx @ success" is the mean prompt context length on solved trajectories.

	Steps / task		Edits / task		Errors enc.		Recovery
Setting	solved	unsolved	solved	unsolved	solved	unsolved	rate (%)
+ R2E-Gym	15.1	22.4	3.3	5.4	3.5	5.8	24.8
+ FIM-Midtrain + R2E-Gym	**23.6**	**32.7**	**7.4**	**10.9**	**6.2**	**8.1**	**28.8**
Output and context summaries							
	Empty patch (% failed)		Loc. correct (% all)		Ctx @ success (tok)		Pass (%)
+ R2E-Gym	3.0		70.6		11,826		26.2
+ FIM-Midtrain + R2E-Gym	**0.3**		**73.4**		**16,397**		**29.2**

hits the step cap on roughly 45% of trajectories, versus 7% for the baseline—but this cost is paid on the unsolved tail; on the solved set the additional steps translate into more correct patches.

D.3 Failure-Mode Breakdown

We label every failed trajectory using a deterministic patch-quality cascade: *no-patch* (output diff is empty); *localization error* (non-empty patch but zero file overlap with the gold patch); *patch error* (file overlap with gold patch but tests fail). Figure 7 reports the outcome distribution: across the three runs, mid-training cuts no-patch failures by an order of magnitude ($\sim 11 \rightarrow \sim 1$ trajectories per run), trims localization errors slightly ($\sim 131 \rightarrow \sim 126$), and leaves the patch-error count essentially flat ($\sim 227 \rightarrow \sim 227$). The $+15$-task gain in solved count is therefore primarily driven by the no-patch mode collapse, supplemented by a small reduction in localization errors.

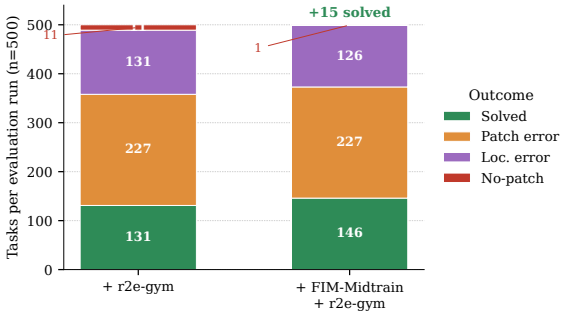

Figure 7: Outcome distribution per evaluation run on SWE-Bench-Verified (14B, R2E-Gym), averaged over three runs. Mid-training collapses the no-patch failure mode ($\sim 11 \rightarrow \sim 1$ tasks per run) and adds $+15$ solved tasks on average; localization and patch errors move only slightly.

D.4 No-Patch Mode: Mechanism

The baseline no-patch failures recovered by mid-training are distributed across all task buckets and are not concentrated in any single repository. In every recovered case the baseline emitted `<finish>` without applying any `str_replace`, while ours applied at least one edit. We interpret this as a direct consequence of the fill-in-the-middle training signal: under FIM the model is always conditioned to produce a non-empty token span between the prefix and suffix, and we observe that this disposition survives the post-training pipeline.

D.5 Multi-File Tasks Are Not Differentially Helped

On the 71 Verified tasks whose gold patch spans ≥ 2 files, both checkpoints solve $\sim 11.3\%$ on average, and per-instance head-to-head on this bucket is balanced. We attribute this to a mismatch in granularity: our function-aware FIM mid-training operates at the function level within files, so cross-file coordination is not directly trained for. Extending the selection algorithm to cross-file function pairs is a natural follow-up.

D.6 A Concrete Contrast

The shift in termination behavior is clearest on cases where the baseline quits prematurely. On `scikit-learn-26323`, for instance, the baseline trajectory runs for a single step in every run, immediately emits `<finish>` with an empty edit and the wrong target file, and is scored as a failure; on the same task our agent runs ~ 41 steps, applies on the order of 20 `str_replace` operations, observes multiple negative tool returns, and recovers to a passing patch. Pooling across the three runs, on the order of ~ 16 of the ~ 55 tasks that ours uniquely solves on Verified have this signature: the baseline emits `<finish>` with zero file overlap with the gold patch (often before any negative observation has even arrived), while ours iterates past intermediate signals and converges on a correct edit. Mid-training does not enable a categorically new capability on these instances; it changes the agent's stopping policy.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.